

Relatório de Entrega — Tech Challenge Fase 2

ToggleMaster — Microserviços, Kubernetes (EKS) e Autoscaling na AWS

Participantes

Nome	RM	Discord
Leonardo Martins Silva	371437	[preencher]

Links de Entrega

Repositório GitHub: <https://github.com/lmartinssilva/togglemaster-fase2>

Vídeo de demonstração: [link a preencher após upload no YouTube/Drive] — duração: 18min52s

Resumo do Projeto

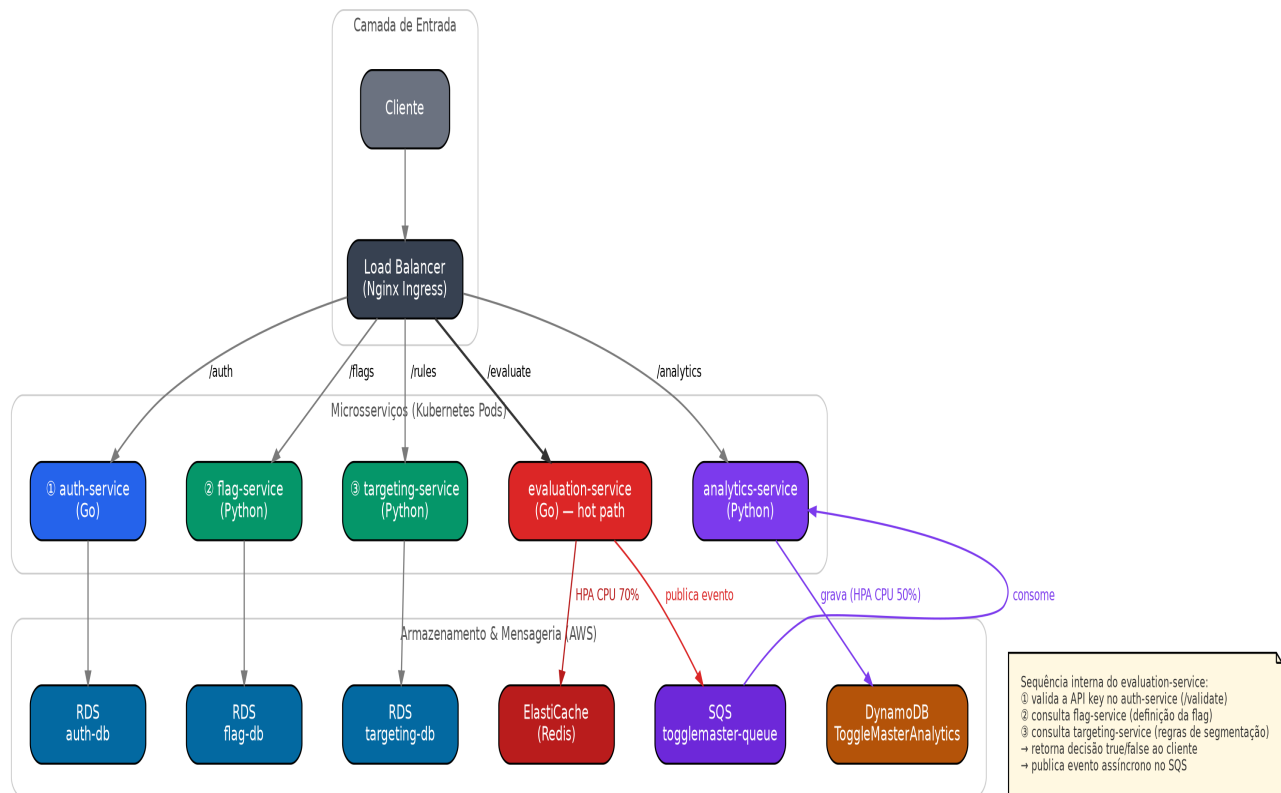
O ToggleMaster evoluiu de um MVP monolítico (Fase 1) para uma arquitetura de 5 microserviços distribuídos, containerizados com Docker e implantados em um cluster Kubernetes gerenciado (AWS EKS), com escalabilidade horizontal automática via HorizontalPodAutoscaler. Toda a infraestrutura foi provisionada manualmente via Console AWS, respeitando as restrições do ambiente AWS Academy (uso exclusivo da LabRole, sem criação de roles IAM novas).

1. Arquitetura — Microserviços

Serviço	Linguagem	Responsabilidade	Persistência
auth-service	Go	Chaves de API e autenticação	PostgreSQL (RDS)
flag-service	Python	CRUD de feature flags	PostgreSQL (RDS)
targeting-service	Python	Regras de segmentação	PostgreSQL (RDS)
evaluation-service	Go	Hot path — decisão true/false	Redis (ElastiCache)
analytics-service	Python	Consumo de eventos e analytics	DynamoDB + SQS

Diagrama de Arquitetura

ToggleMaster — Arquitetura de Microsserviços (Fase 2)



Fluxo de uma avaliação de flag

Cliente → Load Balancer (Nginx Ingress) → evaluation-service → valida chave via auth-service → consulta flag-service / targeting-service (com cache em Redis) → retorna decisão (true/false) → dispara evento assíncrono → SQS → analytics-service → DynamoDB

Diferença entre os Data Stores

RDS (PostgreSQL): dados relacionais estruturados (chaves de API, flags, regras de segmentação), onde consistência e relações entre tabelas importam.

ElastiCache (Redis): cache em memória de baixíssima latência, usado no hot path (evaluation-service) para evitar consultas repetidas ao banco relacional.

DynamoDB: NoSQL, schema flexível e alta capacidade de escrita, adequado ao volume de eventos assíncronos de analytics.

2. Ambiente Local (Docker Compose)

O ecossistema completo foi validado localmente antes da implantação na nuvem, com 9 containers orquestrados via `docker-compose.yml`: os 5 microsserviços, 3 instâncias PostgreSQL (uma por serviço relacional), Redis e DynamoDB Local.

```
docker compose up -d
docker compose ps
```

Todos os health checks passaram com sucesso, confirmando conectividade entre serviços, bancos de dados e cache antes da migração para o ambiente AWS.

3. Infraestrutura AWS Provisionada

- **EKS:** cluster togglemaster-cluster, criado via Console AWS com a LabRole existente, modo Custom configuration (não Auto Mode, incompatível com as restrições do Academy)
- **Node Group:** togglemaster-nodes, instâncias t3.medium, Auto Scaling (mín=1, desejado=2, máx=4), Node IAM Role = LabRole
- **ECR:** 5 repositórios de imagens Docker, um por microserviço, com build multi-stage
- **RDS:** 3 instâncias PostgreSQL independentes (auth-db, flag-db, targeting-db), db.t3.micro
- **ElastiCache:** 1 cluster Redis (cache.t3.micro), usado pelo evaluation-service
- **DynamoDB:** tabela ToggleMasterAnalytics, chave primária event_id (String), billing mode PAY_PER_REQUEST
- **SQS:** fila Standard togglemaster-queue, produzida pelo evaluation-service e consumida pelo analytics-service
- **Nginx Ingress Controller:** instalado via Helm, provisiona automaticamente um Network/Application Load Balancer na AWS, com roteamento por prefixo (/auth, /flags, /rules, /evaluate, /analytics)
- **Metrics Server:** add-on gerenciado nativo do EKS, necessário para o funcionamento do HPA

4. Escalabilidade (HPA)

Como o AWS Academy bloqueia o uso do KEDA (exige criação de roles IAM via IRSA), a escalabilidade foi implementada via HorizontalPodAutoscaler baseado em CPU, conforme sugerido como workaround pelo próprio enunciado do desafio.

HPA	Alvo de CPU	Mín	Max	Observação
evaluation-service-hpa	70%	1	4	Hot path — CPU-bound, escalou conforme esperado sob carga via
analytics-service-hpa	50%	1	4	Calibrado a partir de 70% após observação empírica (ver seção 5)

5. Testes de Carga e Evidências de Funcionamento

5.1 evaluation-service

Teste realizado com Apache Bench, gerando 5.000 requisições com concorrência de 50 simultâneas contra o endpoint /evaluate através do Load Balancer:

```
ab -n 5000 -c 50 "http://<load-balancer>/evaluate/evaluate?user_id=demo&flag_name=demo"
```

Resultado: o HPA reagiu ao aumento de CPU acima de 70%, escalando o número de réplicas do evaluation-service automaticamente, sem intervenção manual. Confirmado via kubectl get pods.

5.2 analytics-service — descoberta e calibração

No teste inicial com o alvo padrão sugerido (70%), o consumo de CPU do analytics-service não ultrapassou ~60%, mesmo com 2.000 mensagens enviadas manualmente à fila SQS em paralelo:

```
cpu: 1%/70% → 10%/70% → 60%/70% → 58%/70% → 57%/70% (patamar, sem escalar)
```

Investigação revelou que o worker do analytics-service processa mensagens de forma sequencial (loop simples), e cada mensagem envolve uma chamada de rede ao DynamoDB — um workload predominantemente I/O-bound, não CPU-bound. Isso explica por que a CPU satura em torno de 55-60% independentemente do volume de mensagens na fila: o gargalo real é latência de rede, não capacidade de processamento.

Decisão de engenharia: ajustado o targetCPUUtilizationPercentage de 70% para 50%, via kubectl patch, com base no comportamento observado empiricamente:

```
kubectl patch hpa analytics-service-hpa -n togglemaster --type='json' \
-p='[{"op":"replace","path":"/spec/metrics/0/resource/target/averageUtilization","value":50}]'
```

Após o ajuste, novos testes de carga confirmaram o HPA escalando corretamente o número de réplicas do analytics-service.

5.3 Pipeline assíncrono SQS → DynamoDB

Validado o fluxo completo de ponta a ponta: requisições HTTP no evaluation-service geraram eventos publicados na fila SQS, consumidos pelo analytics-service e persistidos no DynamoDB. Exemplo de item confirmado via aws dynamodb scan:

```
{"event_id": "8b4f00c2-...", "user_id": "teste", "flag_name": "teste", "result": false,
"timestamp": "2026-07-12T21:08:42Z"}
```

6. Principais Desafios Técnicos

Conflito Auto Mode vs Custom configuration

O EKS Auto Mode tenta usar mecanismos de criação automática de roles IAM, bloqueados no AWS Academy. Solução: seleção explícita de Custom configuration na criação do cluster, com Node Group criado manualmente utilizando a LabRole.

Metrics Server duplicado

A aplicação manual do manifest da comunidade (components.yaml) criou um Deployment concorrente ao add-on gerenciado nativo do EKS, que já existia com labels diferentes das exigidas pelo Service. Resultado: HPA preso em TARGETS <unknown>. Solução: remoção do Deployment manual e resincronização do add-on oficial via `aws eks update-addon --resolve-conflicts OVERWRITE`.

IMDS hop limit

Pods não conseguiam herdar credenciais da LabRole (erro NoCredentialProviders / Unable to locate credentials), pois o limite padrão de saltos de rede do metadata service (IMDS) das instâncias EC2 não cobre o salto adicional originado de dentro de um pod Kubernetes. Solução: `aws ec2 modify-instance-metadata-options --http-put-response-hop-limit 2` em cada nó.

Autenticação entre serviços (SERVICE_API_KEY)

Uma variável de ambiente fixa ("placeholder") no deployments.yaml sobrescrevia o valor real vindo do Secret via envFrom, causando erro 401 em toda chamada entre evaluation-service e flag-service. Diagnosticado comparando o hash SHA-256 esperado com o hash recebido nos logs do auth-service.

Calibração do HPA do analytics-service

Detalhado na seção 5.2 — workload I/O-bound exigiu ajuste do alvo de CPU de 70% para 50% com base em teste empírico de carga.

Renovação de credenciais temporárias do AWS Academy

Sessões de terminal distintas exigem reexportação das variáveis AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY / AWS_SESSION_TOKEN a cada renovação do Lab, por não persistirem entre abas/sessões.

7. Checklist de Entregáveis

Item exigido	Status
Dockerfile otimizado (multi-stage) para os 5 serviços	Concluído
docker-compose.yml com 9 containers locais	Concluído
Cluster EKS via Console (LabRole)	Concluído
5 repositórios ECR + push das imagens	Concluído
3 instâncias RDS PostgreSQL	Concluído

1 cluster ElastiCache Redis	Concluído
1 tabela DynamoDB	Concluído
1 fila SQS	Concluído
Metrics Server instalado	Concluído
Nginx Ingress Controller + Load Balancer	Concluído
Manifestos: Namespace, Deployment, Service, Secret, ConfigMap	Concluído
Ingress com roteamento por prefixo	Concluído
HPA evaluation-service (CPU)	Concluído
HPA analytics-service (CPU, calibrado)	Concluído
Vídeo de demonstração (<20 min)	Concluído — 18min52s
Relatório de entrega (.pdf)	Este documento
Repositório GitHub com código versionado	Concluído

8. Conclusão

O desafio foi concluído com sucesso, cobrindo a totalidade dos requisitos técnicos propostos: containerização, provisionamento manual de infraestrutura AWS sob as restrições do ambiente Academy, orquestração via Kubernetes e escalabilidade automática validada empiricamente sob carga real. Os desafios encontrados ao longo do processo — e resolvidos com investigação sistemática de logs, métricas e comportamento observado — reforçaram o entendimento prático de conceitos centrais de DevOps e arquitetura de microsserviços em ambientes de nuvem restritos.